



Codeus Practice

Unit Tests

Unit Tests: Characteristics

Unit tests are about verifying one small unit of behavior in isolation

Scope

one class or function, one behavior, minimal collaborators

Characteristics

small, fast, automated, isolated

Bonus 0

keeps you business logic valid

Bonus 1

serves as executable documentation for future maintainers

Bonus 2

serves as a great utility to support refactoring



Reference Service for All Examples

JAVA

```
public class DeliveryQuoteService {
    private final RoutePriceClient routePriceClient;
    private final CustomerTierService customerTierService;
    private final QuoteEventPublisher quoteEventPublisher;
    private final Map<String, BigDecimal> quoteCache = new HashMap<>();

    public DeliveryQuoteService(RoutePriceClient routePriceClient,
                               CustomerTierService customerTierService, QuoteEventPublisher quoteEventPublisher) { ... }

    public BigDecimal calculateQuote(String customerId, String destinationCity, BigDecimal parcelWeightKg) {
        validateWeight(parcelWeightKg);
        String cacheKey = buildCacheKey(customerId, destinationCity, parcelWeightKg);
        BigDecimal cachedQuote = quoteCache.get(cacheKey);
        if (cachedQuote != null) {
            return cachedQuote;
        }

        BigDecimal routePricePerKg = routePriceClient.fetchBasePrice(destinationCity);
        String customerTier = customerTierService.tierFor(customerId);
        BigDecimal subtotal = routePricePerKg.multiply(parcelWeightKg);
        BigDecimal total = applyTierDiscount(subtotal, customerTier).setScale(2, RoundingMode.HALF_UP);

        quoteEventPublisher.send(new QuoteCalculatedEvent(customerId, total));
        quoteCache.put(cacheKey, total);
        return total;
    }

    // private buildCacheKey, validateWeight, applyTierDiscount
}

public record QuoteCalculatedEvent(String customerId, BigDecimal total) { }
```



Typical Problem: case 1

- Problem: Hidden Shared State.
- The second test passes only because the first test already filled the private cache.
- It even verifies that collaborators were not called, which hides the order dependency instead of fixing it.

```
JAVA
@TestMethodOrder(OrderAnnotation.class)
class DeliveryQuoteServiceTest {
    static RoutePriceClient routePriceClient = mock(OnlineRoutePriceClient.class);
    static CustomerTierService customerTierService = mock(CustomerTierService.class);
    static QuoteEventPublisher quoteEventPublisher = mock(DeliveryQuoteEventSender.class);
    static DeliveryQuoteService service =
        new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    @Test @Order(1)
    void calculateQuote_cachesGoldQuote() {
        when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
        when(customerTierService.tierFor("c-1")).thenReturn("GOLD");
        assertEquals(new BigDecimal("17.00"), service.calculateQuote("c-1", "Lviv", new BigDecimal("2")));
    }

    @Test @Order(2)
    void calculateQuote_retrievesGoldQuoteFromCache() {
        reset(routePriceClient, customerTierService, quoteEventPublisher);
        assertEquals(new BigDecimal("17.00"), service.calculateQuote("c-1", "Lviv", new BigDecimal("2")));
        verifyNoInteractions(routePriceClient, customerTierService, quoteEventPublisher);
    }
}
```



Typical Problem: case 2

- Problem: Weak Assertions.
- A test is green, but it proves almost nothing about the returned quote.
- If the service returns the wrong number, this test still passes.

JAVA

```
@Test
void calculateQuote_returnsDefaultQuote() {
    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertNotNull(total);
}
```



Typical Problem: case 3

- Problem: Missing Self-Validation.
- The test prints a comparison result instead of failing automatically.
- Someone has to read the console output to know whether behavior is correct.

JAVA

```
@Test
void calculateQuote_returnsDefaultQuote() {
    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));
    boolean passed = total.equals(new BigDecimal("17.00"));

    System.out.println("Expected 17.00, actual = " + total + ", passed = " + passed);
}
```



Typical Problem: case 4

- Problem: Slow External Dependencies.
- This test reaches the network, so it is too slow and environment-sensitive for a unit test.
- The class under test is no longer isolated from infrastructure.

```
JAVA
@Test
void calculateQuote_returnsGoldTierQuoteWithLiveTax() {
    RoutePriceClient routePriceClient = new OnlineRoutePriceClient("https://carrier.example.com");
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    QuoteEventPublisher quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(
        routePriceClient,
        customerTierService,
        quoteEventPublisher
    );

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("17.00"), total);
}
```



Typical Problem: case 5

- Problem: Eager Tests.
- The test looks simple, but it verifies two behaviors at once: quote calculation and event publishing.
- If it fails, the bug could be in pricing, event creation, or event delivery state.

JAVA

```
@Test
void calculateQuote_sendsEventAndReturnsTotal() {
    RoutePriceClient routePriceClient = new LocalRoutePriceClient();
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    DeliveryQuoteEventSender quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("17.00"), total);
    assertEquals(1, quoteEventPublisher.sentEvents().size());
    QuoteCalculatedEvent event = quoteEventPublisher.sentEvents().get(0);
    assertEquals("c-1", event.customerId());
    assertEquals(new BigDecimal("17.00"), event.total());
}
```



Typical Problem: case 6

- Problem: Brittle Reflection Checks.
- This test reaches inside the private cache instead of checking observable behavior.
- A safe refactor of the cache implementation would break the test even if the service still works.

JAVA

```
@Test
void calculateQuote_cachesOnlyOneQuote() throws Exception {
    service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    Field cache = DeliveryQuoteService.class.getDeclaredField("quoteCache");
    cache.setAccessible(true);

    Map<?, ?> values = (Map<?, ?>) cache.get(service);
    assertEquals(1, values.size());
}
```



FIRST Principles

Fast

tests should run quickly enough to be executed constantly

Independent or Isolated

each test should stand on its own and focus on one unit

Repeatable

the same test should produce the same result on any machine and any run

Self-validating

the test should clearly pass or fail without manual inspection

Thorough or Timely

cover important behaviors and edge cases, and write tests at the right time to guide design



FIRST in Practice: Fast

- Replace online and file-based collaborators with mocks or stubs.
- The test becomes quick enough to run on every small change.

JAVA

```
@Test
void calculateQuote_returnsDiscountedTotal_whenDependenciesAreStubbed() {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
    when(customerTierService.tierFor("c-1")).thenReturn("GOLD");

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("17.00"), total);
}
```



FIRST in Practice: Independent or Isolated

- Keep tests independent by creating a fresh service for each test and warming the cache inside the test that needs it.
- The cache setup now happens inside the second test case instead of relying on a previous test run.

```
JAVA

class DeliveryQuoteServiceTest {
    private RoutePriceClient routePriceClient;
    private CustomerTierService customerTierService;
    private QuoteEventPublisher quoteEventPublisher;
    private DeliveryQuoteService service;

    @BeforeEach
    void setUp() {
        routePriceClient = mock(OnlineRoutePriceClient.class);
        customerTierService = mock(CustomerTierService.class);
        quoteEventPublisher = mock(DeliveryQuoteEventSender.class);
        service = new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);
    }

    @Test
    void calculateQuote_returnsDiscountedTotal_whenCustomerHasGoldTier() {
        when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
        when(customerTierService.tierFor("c-1")).thenReturn("GOLD");

        assertEquals(new BigDecimal("17.00"), service.calculateQuote("c-1", "Lviv", new BigDecimal("2")));
    }

    @Test
    void calculateQuote_reusesCachedValue_whenSameRequestIsPreparedInThisTest() {
        when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
        when(customerTierService.tierFor("c-1")).thenReturn("GOLD");

        service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));
        reset(routePriceClient, customerTierService, quoteEventPublisher);
        var cachedResult = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"))

        assertEquals(new BigDecimal("17.00"), cachedResult);
        verifyNoInteractions(routePriceClient, customerTierService, quoteEventPublisher);
    }
}
```



FIRST in Practice: Repeatable

- Repeatable tests control every input from the online client and internal services.
- No network state or previous test runs can change the outcome.

JAVA

```
@Test
void calculateQuote_returnsSameTotal_whenInputsAreControlled() {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
    when(customerTierService.tierFor("c-1")).thenReturn("SILVER");
    DeliveryQuoteService service =
        new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("19.00"), total);
}
```



FIRST in Practice: Self-Validating

- The test should fail loudly and precisely when behavior is wrong.
- Asserting the exact exception message removes guesswork.

```
JAVA
@Test
void calculateQuote_throwsException_whenParcelWeightIsZero() {
    IllegalArgumentException error = assertThrows(
        IllegalArgumentException.class,
        () -> service.calculateQuote("c-1", "Lviv", BigDecimal.ZERO)
    );

    assertEquals("Parcel weight must be positive", error.getMessage());
}
```



FIRST in Practice: Thorough or Timely

- Cover the important branches, not just the happy path.
- A parameterized test keeps discount scenarios visible while the design is still fresh.

JAVA

```
@ParameterizedTest
@CsvSource({
    "GOLD, 17.00" ,
    "SILVER, 19.00" ,
    "STANDARD, 20.00"
})
void calculateQuote_appliesExpectedDiscount_forEachCustomerTier(String tier, String expectedTotal) {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
    when(customerTierService.tierFor("c-1")).thenReturn(tier);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal(expectedTotal), total);
}
```



Other Best Practices: Naming and AAA

- Use descriptive test names such as `method_condition_expectedResult`.
- Keep the test body in Arrange-Act-Assert order.

JAVA

```
@Test
void calculateQuote_returnsGoldPrice_whenCustomerHasGoldTier() {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00")); // Arrange
    when(customerTierService.tierFor("c-1")).thenReturn("GOLD"); // Arrange

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2")); // Act

    assertEquals(new BigDecimal("17.00"), total); // Assert
}
```



Other Best Practices: Focus on Public Behavior

- Verify observable behavior through the public API, not private fields or methods.

JAVA

```
@Test
void calculateQuote_reusesCachedValue_withoutReflection() {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
    when(customerTierService.tierFor("c-1")).thenReturn("GOLD");

    service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));
    service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    verify(routePriceClient, times(1)).fetchBasePrice("Lviv");
    verify(quoteEventPublisher, times(1)).send(any(QuoteCalculatedEvent.class));
}
```



Other Best Practices: Focus on One Behavior

- Prefer one behavior per test so each failure has one clear cause.

JAVA

```
@Test
void calculateQuote_sendsEventAndReturnsTotal() {
    RoutePriceClient routePriceClient = new LocalRoutePriceClient();
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    DeliveryQuoteEventSender quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    // fails only if 'total' is calculated incorrectly
    assertEquals(new BigDecimal("17.00"), total);
}

@Test
void calculateQuote_sendsEvent() {
    RoutePriceClient routePriceClient = new LocalRoutePriceClient();
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    DeliveryQuoteEventSender quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    // fails mainly if 'event' is not sent or contains incorrect data
    assertNotNull(total);
    assertEquals(1, quoteEventPublisher.sentEvents().size());
    QuoteCalculatedEvent event = quoteEventPublisher.sentEvents().get(0);
    assertEquals("c-1", event.customerId());
    assertEquals(new BigDecimal("17.00"), event.total());
}
```





Thank you